

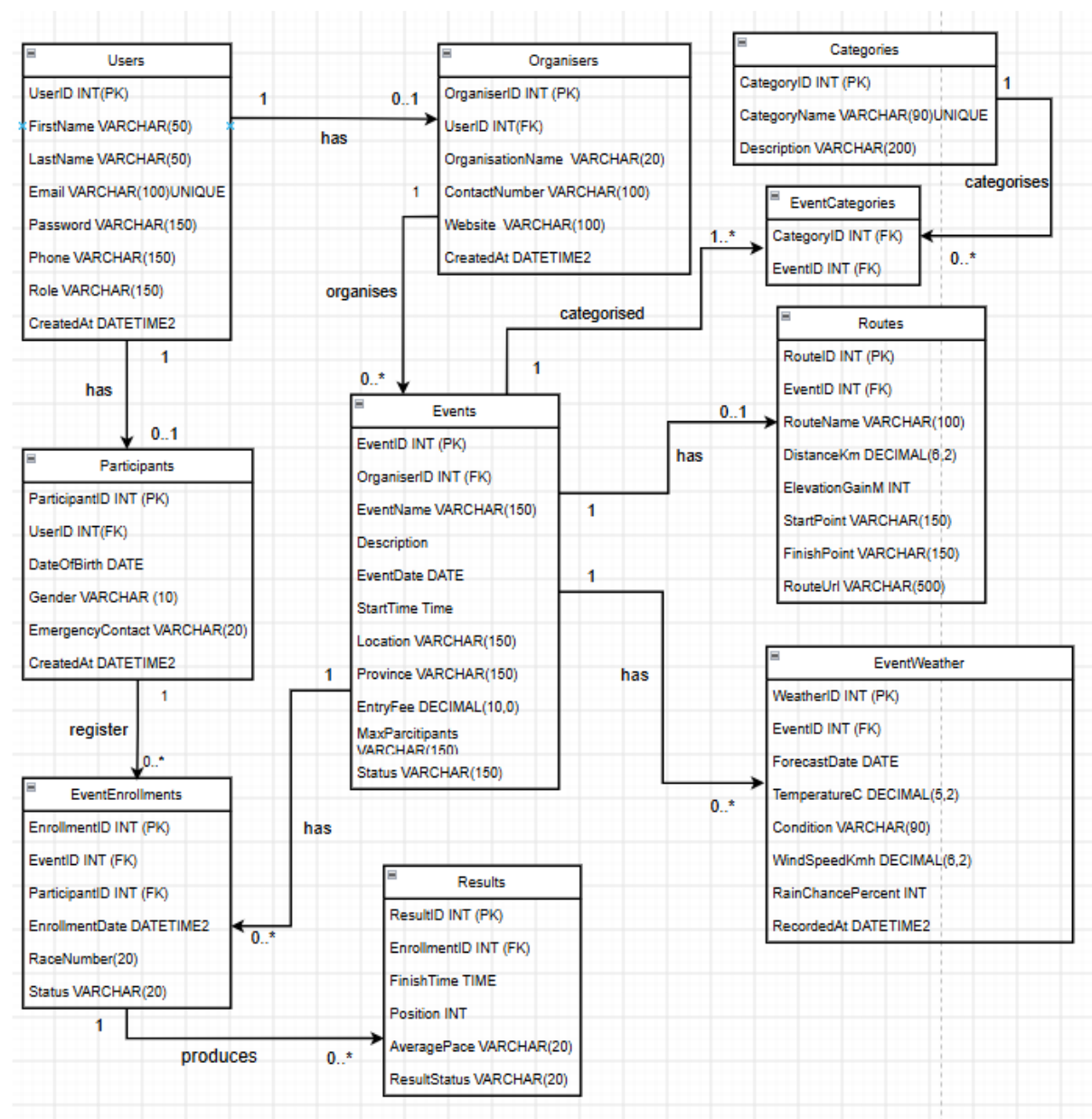
Khutso Pollen
Masekwameng
ST10097395
Progarmming 2B
Assignment 1

Youtube link : <https://youtu.be/yzA4btuLMLU>

Github link: <https://github.com/KhutsoPollenMasekwameng/Prog-2b-Assignment1-part1.git>

Question 1 – Section A

ERD :



Business Rules

- A User may have zero or one Organiser profile, and each Organiser belongs to exactly one User.
- A User may have zero or one Participant profile, and each Participant belongs to exactly one User.
- An Organiser may organise many Events, but each Event is organised by exactly one Organiser.
- A Participant may register for many Events, and an Event may have many Participants.
- A Participant may register for a particular Event only once.
- An Event must have at least one Category, while a Category may be associated with many Events.
- An Event cannot be assigned the same Category more than once.
- An Event may have zero or one Route, and each Route belongs to exactly one Event.
- An Event may have many Weather records, while each Weather record belongs to exactly one Event.
- An Event may have many EventEnrollments, while each EventEnrollment belongs to exactly one Event.
- Each EventEnrollment belongs to exactly one Participant.
- An Event cannot have more Participants than its MaxParticipants value.
- A Participant's RaceNumber must be unique within an Event.
- An EventEnrollment may have zero or one Result, while each Result belongs to exactly one EventEnrollment.
- All foreign-key values must reference an existing record to maintain referential integrity.

Question 2 - API Endpoint Plan

1. Event Organiser API Endpoints:

HTTP method	Route	Description	Role required	Request body	Expected response
POST	/api/auth/register	Creates a new Event Organiser account.	Event Organiser	{ name, email,	201 Created – account created. 400 Bad Request –

				password , role }	invalid details. 409 Conflict – email already exists.
POST	/api/auth/login	Authenticat es an Event Organiser and provides an authenticati on token.	Event Organis er	{ email, password }	200 OK – login successful with token. 401 Unauthorized – invalid credentials.
GET	/api/users/me	Retrieves the logged- in organiser's profile information.	Event Organis er	None	200 OK – organiser profile. 401 Unauthorized – authentication required.
PUT	/api/users/me	Updates the logged-in organiser's profile information.	Event Organis er	{ name, email, phone, location }	200 OK – profile updated. 400 Bad Request – invalid information.
GET	/api/categories	Retrieves all event categories available when creating an event.	Event Organis er	None	200 OK – list of categories.
POST	/api/categories	Creates a new event category.	Event Organis er	{ name, descriptio n }	201 Created – category created. 400 Bad Request – invalid data.
PUT	/api/categories/{id}	Updates an existing event category.	Event Organis er	{ name, descriptio n }	200 OK – category updated. 404 Not Found – category does not exist.
DELETE	/api/categories/{id}	Deletes an event category that is no	Event Organis er	None	204 No Content – category deleted. 404 Not Found –

		longer required.			category does not exist.
GET	/api/events	Retrieves events managed by the organiser.	Event Organiser	None	200 OK – list of organiser events.
GET	/api/events/{id}	Retrieves detailed information about a specific event.	Event Organiser	None	200 OK – event details. 404 Not Found – event does not exist.
POST	/api/events	Creates and publishes a new running, walking or cycling event.	Event Organiser	{ name, description, categoryId, date, startTime, location, distance, routeInfo }	201 Created – event created. 400 Bad Request – invalid data.
PUT	/api/events/{id}	Updates information for an event managed by the organiser.	Event Organiser	{ name, description, categoryId, date, startTime, location, distance, routeInfo }	200 OK – event updated. 404 Not Found – event does not exist. 403 Forbidden – organiser does not own event.
DELETE	/api/events/{id}	Removes an event managed by the organiser.	Event Organiser	None	204 No Content – event deleted. 404 Not Found – event does not exist.
GET	/api/events/{id}/enrolments	Retrieves all participants enrolled in a specific event.	Event Organiser	None	200 OK – list of enrolled participants. 404 Not Found – event does not exist.
POST	/api/events/{id}/results	Records a participant's result after an event has	Event Organiser	{ userId, finishTime, position, status }	201 Created – result recorded. 400 Bad Request –

		been completed.			invalid result. 404 Not Found – participant/event not found.
GET	/api/events/{id}/results	Retrieves the results for all participants in an event.	Event Organizer	None	200 OK – event results. 404 Not Found – event does not exist.
PUT	/api/results/{id}	Corrects or updates a participant's result.	Event Organizer	{ finishTime , position, status }	200 OK – result updated. 404 Not Found – result does not exist.

2. Participant API Endpoints:

HTTP method	Route	Description	Role Required	Request Body	Expected response
POST	/api/auth/register	Creates a new Participant account so the user can enrol in events and track performance.	Participant	{ name, email, password, role }	201 Created – account created. 400 Bad Request – invalid details. 409 Conflict – email already exists.
POST	/api/auth/login	Authenticates a Participant and provides an authentication token.	Participant	{ email, password }	200 OK – login successful with token. 401 Unauthorized – invalid credentials.
GET	/api/users/me	Retrieves the logged-in	Participant	{email,password}	200 OK – participant

		participant's profile information.			profile. 401 Unauthorized – authentication required.
PUT	/api/users/me	Updates the logged-in participant's profile information.	Participant	{ name, email, phone, location }	200 OK – profile updated. 400 Bad Request – invalid information.
GET	/api/categories	Retrieves available event categories such as running, walking and cycling.	Participant	None	200 OK – list of categories.
GET	/api/events	Allows participants to browse upcoming events.	Participant	None	200 OK – list of upcoming events.
GET	/api/events/{id}	Displays detailed information about a selected event.	Participant	None	200 OK – event details. 404 Not Found – event does not exist.
POST	/api/events/{id}/enrolments	Registers the logged-in participant for a specific event.	Participant	{ emergencyContact, selectedDistance }	201 Created – enrolment created. 404 Not Found – event does not exist. 409 Conflict – already enrolled.

GET	/api/enrolments	Retrieves all events in which the logged-in participant is enrolled.	Participant	None	200 OK – participant's enrolments.
DELETE	/api/enrolments/{id}	Allows a participant to cancel an event enrolment.	Participant	None	200 OK – enrolment details. 404 Not Found – enrolment does not exist.
GET	/api/users/me/results	Retrieves the participant's complete personal performance history.	Participant	None	200 OK – performance history. 401 Unauthorized – authentication required.
GET	/api/events/{id}/weather	Provides weather information for the event location to help the participant prepare for race day.	Participant	None	200 OK – participant results.

Question 3

SQL CODE:

CREATE DATABASE RaceDay;

=====

-- 1. USERS

```

-----

CREATE TABLE Users
(
    UserID INT IDENTITY(1,1) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    Password VARCHAR(150) NOT NULL,
    Phone VARCHAR(150) NOT NULL,
    Role VARCHAR(150) NOT NULL
    CONSTRAINT DF_Users_Role DEFAULT ('Participant'),
    CreatedAt DATETIME2 NOT NULL
    CONSTRAINT DF_Users_CreatedAt DEFAULT (SYSDATETIME()),

    CONSTRAINT PK_Users PRIMARY KEY (UserID),
    CONSTRAINT UQ_Users_Email UNIQUE (Email)
);

```

```

-----

--ORGANISERS

-----

```

```

CREATE TABLE Organisers
(
    OrganiserID INT IDENTITY(1,1) NOT NULL,
    UserID INT NOT NULL,

```

```

OrganisationName VARCHAR(20) NOT NULL,
ContactNumber VARCHAR(100) NOT NULL,
Website VARCHAR(100) NULL,
CreatedAt DATETIME2 NOT NULL
    CONSTRAINT DF_Organisers_CreatedAt DEFAULT (SYSDATETIME()),

CONSTRAINT PK_Organisers PRIMARY KEY (OrganiserID),


CONSTRAINT UQ_Organisers_UserID UNIQUE (UserID),


CONSTRAINT FK_Organisers_Users
    FOREIGN KEY (UserID)
    REFERENCES Users(UserID)
);

=====

--participants

=====


CREATE TABLE Participants
(
    ParticipantID INT IDENTITY(1,1) NOT NULL,
    UserID INT NOT NULL,
    DateOfBirth DATE NOT NULL,
    Gender VARCHAR(10) NOT NULL,
    EmergencyContact VARCHAR(20) NOT NULL,

```

CreatedAt DATETIME2 NOT NULL

CONSTRAINT DF_Participants_CreatedAt DEFAULT (SYSDATETIME()),

CONSTRAINT PK_Participants PRIMARY KEY (ParticipantID),

-- One user can have at most one participant profile

CONSTRAINT UQ_Participants_UserID UNIQUE (UserID),

CONSTRAINT FK_Participants_Users

FOREIGN KEY (UserID)

REFERENCES Users(UserID)

);

=====

-- 4. EVENTS

-- =====

CREATE TABLE Events

(

EventID INT IDENTITY(1,1) NOT NULL,

OrganiserID INT NOT NULL,

EventName VARCHAR(150) NOT NULL,

Description VARCHAR(500) NULL,

EventDate DATE NOT NULL,

StartTime TIME NOT NULL,

Location VARCHAR(150) NOT NULL,

Province VARCHAR(150) NOT NULL,

EntryFee DECIMAL(10,2) NOT NULL

```

        CONSTRAINT DF_Events_EntryFee DEFAULT (0.00),
MaxParticipants VARCHAR(150) NOT NULL,
Status VARCHAR(150) NOT NULL
        CONSTRAINT DF_Events_Status DEFAULT ('Upcoming'),

CONSTRAINT PK_Events PRIMARY KEY (EventID),

CONSTRAINT FK_Events_Organisers
    FOREIGN KEY (OrganiserID)
    REFERENCES Organisers(OrganiserID),

CONSTRAINT CK_Events_EntryFee
    CHECK (EntryFee >= 0)
);

=====
-- 5. CATEGORIES
=====

CREATE TABLE Categories
(
    CategoryID INT IDENTITY(1,1) NOT NULL,
    CategoryName VARCHAR(90) NOT NULL,
    Description VARCHAR(200) NULL,

    CONSTRAINT PK_Categories PRIMARY KEY (CategoryID),
    CONSTRAINT UQ_Categories_CategoryName UNIQUE (CategoryName)

```

);

```
=====
-- 6. EVENT CATEGORIES
=====
```

```
CREATE TABLE EventCategories
(
    CategoryID INT NOT NULL,
    EventID INT NOT NULL,

    CONSTRAINT PK_EventCategories
        PRIMARY KEY (CategoryID, EventID),

    CONSTRAINT FK_EventCategories_Categories
        FOREIGN KEY (CategoryID)
        REFERENCES Categories(CategoryID),

    CONSTRAINT FK_EventCategories_Events
        FOREIGN KEY (EventID)
        REFERENCES Events(EventID)
);
```

```
=====
-- 7. ROUTES
=====
```

```
CREATE TABLE Routes
(
    RouteID INT IDENTITY(1,1) NOT NULL,
    EventID INT NOT NULL,
    RouteName VARCHAR(100) NOT NULL,
    DistanceKm DECIMAL(6,2) NOT NULL,
    ElevationGainM INT NOT NULL,
    StartPoint VARCHAR(150) NOT NULL,
    FinishPoint VARCHAR(150) NOT NULL,
    RouteURL VARCHAR(500) NULL,

    CONSTRAINT PK_Routes PRIMARY KEY (RouteID),

    CONSTRAINT UQ_Routes_EventID UNIQUE (EventID),

    CONSTRAINT FK_Routes_Events
        FOREIGN KEY (EventID)
        REFERENCES Events(EventID),

    CONSTRAINT CK_Routes_Distance
        CHECK (DistanceKm > 0),

    CONSTRAINT CK_Routes_Elevation
        CHECK (ElevationGainM >= 0)
);
```

```
--=====
-- 8. EVENT ENROLLMENTS
--=====
```

```
CREATE TABLE EventEnrollments
(
    EnrollmentID INT IDENTITY(1,1) NOT NULL,
    EventID INT NOT NULL,
    ParticipantID INT NOT NULL,
    EnrollmentDate DATETIME2 NOT NULL
        CONSTRAINT DF_EventEnrollments_EnrollmentDate
        DEFAULT (SYSDATETIME()),
    RaceNumber VARCHAR(20) NOT NULL,
    Status VARCHAR(20) NOT NULL
        CONSTRAINT DF_EventEnrollments_Status
        DEFAULT ('Registered'),

    CONSTRAINT PK_EventEnrollments
        PRIMARY KEY (EnrollmentID),

    CONSTRAINT FK_EventEnrollments_Events
        FOREIGN KEY (EventID)
        REFERENCES Events(EventID),

    CONSTRAINT FK_EventEnrollments_Participants
        FOREIGN KEY (ParticipantID)
```

REFERENCES Participants(ParticipantID),

CONSTRAINT UQ_EventEnrollments_EventParticipant

UNIQUE (EventID, ParticipantID)

);

=====

-- 9. RESULTS

=====

CREATE TABLE Results

(

ResultID INT IDENTITY(1,1) NOT NULL,

EnrollmentID INT NOT NULL,

FinishTime TIME NULL,

Position INT NULL,

AveragePace VARCHAR(20) NULL,

ResultStatus VARCHAR(20) NOT NULL

CONSTRAINT DF_Results_ResultStatus

DEFAULT ('Pending'),

CONSTRAINT PK_Results PRIMARY KEY (ResultID),

CONSTRAINT UQ_Results_EnrollmentID UNIQUE (EnrollmentID),

```
CONSTRAINT FK_Results_EventEnrollments
    FOREIGN KEY (EnrollmentID)
    REFERENCES EventEnrollments(EnrollmentID),
```

```
CONSTRAINT CK_Results_Position
    CHECK (Position IS NULL OR Position > 0)
```

```
);
```

```
=====
-- 10. EVENT WEATHER
=====
```

```
CREATE TABLE EventWeather
(
    WeatherID INT IDENTITY(1,1) NOT NULL,
    EventID INT NOT NULL,
    ForecastDate DATE NOT NULL,
    TemperatureC DECIMAL(5,2) NOT NULL,
    Condition VARCHAR(80) NOT NULL,
    WindSpeedKph DECIMAL(6,2) NOT NULL,
    RainChancePercent INT NOT NULL,
    RecordedAt DATETIME2 NOT NULL
    CONSTRAINT DF_EventWeather_RecordedAt
        DEFAULT (SYSDATETIME()),

    CONSTRAINT PK_EventWeather PRIMARY KEY (WeatherID),
```

CONSTRAINT FK_EventWeather_Events

FOREIGN KEY (EventID)

REFERENCES Events(EventID),

CONSTRAINT CK_EventWeather_WindSpeed

CHECK (WindSpeedKph >= 0),

CONSTRAINT CK_EventWeather_RainChance

CHECK (RainChancePercent BETWEEN 0 AND 100)

);

--=====

-- SELECT

-- =====

INSERT INTO Users

(FirstName, LastName, Email, Password, Phone, Role)

VALUES

('Thabo', 'Mokoena',

'thabo.mokoena@raceday.co.za',

'Password123!',

'0825551001',

'Organiser'),

('Naledi', 'Dlamini',

'naledi.dlamini@raceday.co.za',

'Password123!',

'0835551002',

'Organiser'),

('lad', 'chant',

'pollen.masekwameng@email.com',

'Password123!',

'0845551003',

'Participant'),

('Lerato', 'Molefe',

'lerato.molefe@email.com',

'Password123!',

'0855551004',

'Participant');

INSERT INTO Organisers

(UserID, OrganisationName, ContactNumber, Website)

VALUES

(1,

'RunSA',

'0115551001',

'https://www.runsa.co.za'),

(2,

'ActiveRace',

```
'0115551002',  
'https://www.activerace.co.za');
```

INSERT INTO Participants

(UserID, DateOfBirth, Gender, EmergencyContact)

VALUES

```
(3,  
'2000-05-15',  
'Male',  
'0795552001'),
```

```
(4,  
'1999-08-22',  
'Female',  
'0795552002');
```

INSERT INTO Events

(OrganiserID, EventName, Description, EventDate, StartTime,
Location, Province, EntryFee, MaxParticipants, Status)

VALUES

```
(1,  
'Johannesburg City Run',  
'A competitive road running event through central Johannesburg.',
```

'2026-10-10',
'07:00:00',
'Johannesburg',
'Gauteng',
150.00,
'500',
'Upcoming'),

(1,
'Pretoria Spring Race',
'A community road race suitable for beginner and experienced runners.',
'2026-10-24',
'06:30:00',
'Pretoria',
'Gauteng',
120.00,
'400',
'Upcoming'),

(2,
'Cape Town Coastal Run',
'A scenic coastal running event with ocean views.',
'2026-11-07',
'06:00:00',
'Cape Town',
'Western Cape',
200.00,
'600',

'Upcoming');

INSERT INTO Categories

(CategoryName, Description)

VALUES

('5 KM',

'Five kilometre running category.'),

('10 KM',

'Ten kilometre running category.'),

('21 KM',

'Half marathon category.');

INSERT INTO EventCategories

(CategoryID, EventID)

VALUES

(1, 1),

(2, 1),

(3, 1),

(1, 2),

(2, 2),

(3, 2),

(1, 3),

(2, 3),

(3, 3);

INSERT INTO Routes

(EventID, RouteName, DistanceKm, ElevationGainM,

StartPoint, FinishPoint, RouteURL)

VALUES

(1,

'Johannesburg Central Route',

10.00,

120,

'Nelson Mandela Square',

'Sandton Sports Club',

'https://www.runsa.co.za/routes/jhb'),

(2,

'Pretoria Union Buildings Route',

10.00,

150,

'Union Buildings',

'Pretoria National Botanical Garden',

'https://www.runsa.co.za/routes/pta'),

(3,

'Cape Town Coastal Route',

21.10,

```
180,  
'Sea Point',  
'Camps Bay',  
'https://www.activerace.co.za/routes/cpt');
```

```
INSERT INTO EventEnrollments
```

```
(EventID, ParticipantID, RaceNumber, Status)
```

```
VALUES
```

```
(1, 1, 'JHB001', 'Registered'),
```

```
(1, 2, 'JHB002', 'Registered'),
```

```
(2, 1, 'PTA001', 'Registered'),
```

```
(3, 2, 'CPT001', 'Registered');
```

```
INSERT INTO Results
```

```
(EnrollmentID, FinishTime, Position, AveragePace, ResultStatus)
```

```
VALUES
```

```
(1,
```

```
'00:52:35',
```

```
12,
```

```
'05:15 min/km',
```

'Completed'),

(2,

'00:58:42',

25,

'05:52 min/km',

'Completed');

INSERT INTO EventWeather

(EventID, ForecastDate, TemperatureC, Condition,

WindSpeedKph, RainChancePercent)

VALUES

(1,

'2026-10-10',

18.50,

'Sunny',

12.50,

10),

(2,

'2026-10-24',

20.00,

'Partly Cloudy',

15.20,

20),

(3,
'2026-11-07',
19.50,
'Clear',
18.00,
15);